

```

1 // ===== CLIENT.H =====
2 #include<sys/types.h>
3 #include<sys/socket.h>
4 #include<netdb.h>
5
6 // ----- CONNECT TO WEBSITE -----
7 int connect_to_website(char *website_addr){
8     struct addrinfo hints;
9     struct addrinfo *servinfo;
10    int status_getaddrinfo;
11    int client_socket;
12
13    memset(&hints, 0, sizeof hints);
14    hints.ai_socktype = SOCK_STREAM; // TCP
15    hints.ai_family = AF_INET6; // IPv6
16
17
18    status_getaddrinfo = getaddrinfo(website_addr, "80", &hints, &servinfo);
19    if(status_getaddrinfo != 0){
20        fprintf(stderr, "Eroare la optinerea adresei: %s \n",
21            gai_strerror(status_getaddrinfo));
22        return 0;
23    }
24
25    client_socket = socket(servinfo->ai_family, servinfo->ai_socktype,
26        servinfo->ai_protocol);
27    if(client_socket == -1){
28        perror(" -> Socket-ul clientului NU a fost deschis! \n");
29        return 0;
30    }
31    puts(" -> Socket-ul clientului a fost deschis");
32
33    if(connect(client_socket, servinfo->ai_addr, servinfo->ai_addrlen) == -1){
34        perror(" -> Eroare la conectare! \n");
35        return 0;
36    }else{
37        puts(" -> Conectarea a fost realizata cu succes! ");
38        return client_socket;
39    }
40 }
41 // ----- END FUNCTION -----
42
43 // ----- SEND MSG TO WEBSITE -----
44 void send_msg_to_website(int sock, char * http_msg){
45     int length_msg = strlen(http_msg);
46     if(send(sock, http_msg, length_msg, 0) == -1){
47         perror(" -> Mesajul catre web server NU a fost trimis \n");
48     }else{
49         puts(" -> Mesajul catre web server a fost trimis!");
50     }
51 }
52 // ----- END FUNCTIN -----
53
54 // ----- RECV WEBSITE PAGE -----
55 void recv_website(int client_sock, int accept_sock, char *file_name){
56     int lenght_data;
57     char data_receive[2000];
58     FILE *webPage;
59     int nr_rcv = 0;
60     webPage = fopen(file_name, "w");
61     do{
62         lenght_data = recv(client_sock, data_receive, 2000, 0);
63         if(lenght_data == -1){
64             perror(" -> Eroare la primirea dateleor \n");
65             break;
66         }
67     }

```

```
68         nr_recv++;
69         fprintf(stderr, " -> Datele %d au fost primite!\n", nr_recv);
70         send(accept_sock, data_receive, 2000, 0);
71         fwrite(data_receive, lenght_data, 1, webPage);
72     }while(lenght_data != 0);
73
74     fclose(webPage);
75 }
76 // ----- END FUNCTION -----
77
```

```

1  // ===== MAIN =====
2  #include<string.h>
3  #include<stdio.h>
4
5  #include "server.h"
6  #include "client.h"
7
8  #define PORT 22218
9  #define PENDING_CON 0
10 #define COMMAND 16
11 #define WEBSITE "www.traceroute6.net"
12
13 #define HTTP_MSG "GET / HTTP/1.0\r\n\r\n"
14 #define FILENAME "index.html"
15
16 int main(){
17     char buffer_data_rcv[100];
18     int data_rcv;
19     char web_page[2000];
20
21     int client_socket;
22     int server_socket;
23
24     // Open Socket
25     server_socket = open_server_socket();
26     // Bind Socket
27     bind_server_socket(server_socket, PORT);
28     // Listen
29     start_listen(server_socket, PENDING_CON);
30
31     while(1){
32         // Acceptare conexiune
33         int accept_socket = accept_connections(server_socket);
34         if(accept_socket == 0){
35             continue;
36         }
37         do{
38             data_rcv = recv(accept_socket, buffer_data_rcv, sizeof buffer_data_rcv, 0);
39             if(!CheckCommand(buffer_data_rcv, COMMAND)){
40                 puts(" -> Comanda neimplementata!");
41                 send(accept_socket, "Comanda neimplementata! \n", 100, 0);
42                 continue;
43             }else if(data_rcv > 0){
44                 puts(" -> Comanda implementata ");
45                 //Open and connect socket
46                 client_socket = connect_to_website(WEBSITE);
47                 //Trimite mesaj care adresa web
48                 send_msg_to_website(client_socket, HTTP_MSG);
49                 //Mesajul primit se trimite in aplicatia clientului si intr-un fisier
50                 recv_website(client_socket, accept_socket, FILENAME);
51                 //Close socket
52                 close(client_socket);
53             }
54         }while(data_rcv > 0);
55         close(accept_socket);
56         puts(" -> Client deconectata! \n ");
57     }
58
59     close(server_socket);
60     puts(" -> Socket-ul serverului a fost inchis");
61     return 0;
62 }
63

```

```

1 // ===== SERVER.H =====
2 #include<sys/types.h>
3 #include<sys/socket.h>
4 #include<netdb.h>
5
6 // ----- OPEN SERVER SOCKET FUNCTION -----
7 int open_server_socket(){
8     int server_socket;
9     int true = 1;
10
11     // Deschidere socket
12     // AF_INET - IPv4
13     // SOCK_STREAM - conectare TCP
14     server_socket = socket(AF_INET, SOCK_STREAM, 0);
15     if(server_socket == -1){
16         perror(" -> Socket-ul nu a fost deschis! \n");
17         return 0;
18     }
19     puts(" -> Socket-ul a fost deschis!");
20
21     // Optiune Socket
22     // - server_socket - socketul utilizat
23     // - SOL_SOCKET - pentru cine ii definita optiunea
24     // - SO_REUSEADDR - permite socket-ului sa faca bind la o adresa si port deja
    utilizate
25     if(setsockopt(server_socket, SOL_SOCKET, SO_REUSEADDR, &true, sizeof(int)) == -1){
26         perror(" -> Eroare la setarea optiuni \n");
27         return 0;
28     }
29
30     return server_socket;
31 }
32
33 // ----- END FUNCTION -----
34
35 // ----- BIND SERVER SOCKET FUCNTION -----
36 void bind_server_socket(int sock, int port){
37     struct sockaddr_in serv_addr;
38
39     serv_addr.sin_family = AF_INET; // IPv4
40     serv_addr.sin_port = htons(port); // port
41     serv_addr.sin_addr.s_addr = INADDR_ANY; //selectare automata a interfecei
42
43     if(bind(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) == -1){
44         perror(" -> Eroare bind socket! \n");
45     }else{
46         printf(" -> Bind OK pe portul %d \n ", port);
47     }
48
49 }
50 // ----- END FUNCTION -----
51
52 // ----- START LISTEN FUNCTION -----
53 void start_listen(int sock, int pending_con){
54     if(listen(sock, pending_con) == -1){
55         perror(" -> Eroare asteptare client! \n");
56     }else{
57         puts("\n Asteptare client ... \n");
58     }
59
60 }
61 // ----- END FUNCTION -----
62
63 // ----- ACCEPT CONNECTIONS FUNCTION -----
64 int accept_connections(int sock){
65     struct sockaddr_storage their_addr;
66     socklen_t addr_size;
67
68     addr_size = sizeof their_addr;

```

```

69     int accept_socket = accept(sock, (struct sockaddr *)&their_addr, &addr_size);
70     if(accept_socket == -1){
71         perror(" -> Conexiunea NU a fost acceptata \n");
72         send(accept_socket, "Conexiunea NU a fost acceptata!\n", 100, 0);
73         return 0;
74     }else{
75         puts(" -> Conexiune acceptata! ");
76         send(accept_socket, "Conexiunea a fost acceptata! \n", 100, 0);
77         return accept_socket;
78     }
79 }
80 // ----- END FUNCTION -----
81
82
83 // ----- CHECK COMMAND -----
84 int CheckCommand(char *buffer, char *command){
85
86     if(buffer[0] == command[0] && buffer[1] == command[1] && buffer[2] == '#'){
87         return 1;
88     }
89     else{
90         return 0;
91     }
92
93 }
94 // ----- END FUNCTION -----
95

```